BLOCKSEC

Security Audit Report for Hook Contracts

Date: October 23, 2026 **Version:** 1.0

Contact: contact@blocksec.com

Contents

Chapter 1 Introduction **1**

1.1 About the Audit Target . . . 1
1.2 Disclaimer . . . 1
1.3 Procedure of Auditing . . . 2
1.3.1 Security Issues . . . 2
1.3.2 Additional Recommendation . . . 3
1.4 Security Model . . . 3

Chapter 2 Findings **4**

2.1 Security Issue . . . 4
2.1.1 Potential DoS due to a zero output amount in buy orders . . . 4
2.1.2 Non-standard implementation of the assembly code in the `_delegateModule()` function . . . 5
2.1.3 Inconsistent capacity check in the function `beforeSwap()` . . . 6
2.2 Recommendation . . . 7
2.2.1 Emit events in the function `setQuoter()` . . . 7
2.2.2 Remove redundant code . . . 7
2.2.3 Add validity checks for the input `quoter_` in the function `setQuoter()` . . . 7
2.3 Note . . . 8
2.3.1 Potential centralization risks . . . 8

Report Manifest

Item	Description
Client	BigShort
Target	Hook Contracts

Version History

Version	Date	Description
1.0	October 23, 2026	First release

Signature



Digitally signed by: BlockSec (https://blocksec.com)

SHA1 digest of public key: BAB106D3708BC6DE0963CE05E9478C24B326AE1C

Date: 2026-10-26 03:18:49+0000

About BlockSec BlockSec focuses on the security of the blockchain ecosystem and collaborates with leading DeFi projects to secure their products. BlockSec is founded by top-notch security researchers and experienced experts from both academia and industry. They have published multiple blockchain security papers in prestigious conferences, reported several zero-day attacks of DeFi applications, and successfully protected digital assets that are worth more than 14 million dollars by blocking multiple attacks. They can be reached at Email, Twitter and Medium.

Chapter 1 Introduction

1.1 About the Audit Target

Information	Description
Type	Smart Contract
Language	Solidity
Approach	Semi-automatic and manual verification

The audit target (hereinafter referred to as the Target) of this audit is the code repository [1] of Hook Contracts of BigShort.

Hook Contracts of BigShort are built on top of the PancakeSwap Infinity protocol and serve as the trading and settlement layer in BigShort's LaunchPad Protocol. Specifically, the contract BigShortV2PolicyHook handles callbacks of the Pancake Infinity pools and enforces that all swaps must be executed via the official router contract registered in the Pancake Infinity protocol. Moreover, each swap triggers tax collection and fund settlement. These collected taxes are then allocated to referrers and the FOMO prize pool. Once a buy order reaches the defined terminal price, the graduation is triggered by adding a full-range position using the existing liquidity.

Note this audit only focuses on the codes located in the following directories/files.

- src/v2/BigShortV2PolicyHook.sol
- src/v2/BigShortV2RevenueModule.sol
- src/v2/BigShortV2SettlementModule.sol

Other files are not within the scope of the audit. Additionally, all dependencies of the Target are considered reliable in terms of both functionality and security, and are therefore not included in the audit scope.

The auditing process is iterative. Specifically, we would audit the commits that fix the discovered issues. If there are new issues, we will continue this process. The commit SHA values during the audit are shown in the following table. Our audit report is responsible for the code in the initial version (`Version 1`), as well as new code (in the following versions) to fix issues in the audit report. Code prior to and including the baseline version (`Version 0`), where applicable, is outside the scope of this audit and assumes to be reliable and secure.

Project	Version	Commit Hash
BigshortProtocol-EVM-v2	`Version 1`	`18037466692dbaaa98cdf2114fa798de2663a473`
	`Version 2`	`65cc203e252460bc382cf96921230a60eed8c9f6`

1.2 Disclaimer

This audit report does not constitute investment advice or a personal recommendation. It does not consider, and should not be interpreted as considering or having any bearing on,

[1] https://github.com/BigShortLTD/BigshortProtocol-EVM-v2.git

the potential economics of a token, token sale or any other product, service or other asset. Any entity should not rely on this report in any way, including for the purpose of making any decisions to buy or sell any token, product, service or other asset.

This audit report is not an endorsement of any particular project or team, and the report does not guarantee the security of any particular project. This audit does not give any warranties on discovering all security issues of the Target, i.e., the evaluation result does not guarantee the nonexistence of any further findings of security issues. As one audit cannot be considered comprehensive, we always recommend proceeding with independent audits and a public bug bounty program to ensure the security of the Target.

The scope of this audit is limited to the code mentioned in Section 1.1. Unless explicitly specified, the security of the language itself (e.g., the solidity language), the underlying compiling toolchain and the computing infrastructure are out of the scope.

1.3 Procedure of Auditing

We perform the audit according to the following procedure.

- **Vulnerability Detection** We first scan the Target with automatic code analyzers, and then manually verify (reject or confirm) the issues reported by them.
- **Semantic Analysis** We study the business logic of the Target and conduct further investigation on the possible vulnerabilities using an automatic fuzzing tool (developed by our research team). We also manually analyze possible attack scenarios with independent auditors to cross-check the result.
- **Recommendation** We provide some useful advice to developers from the perspective of good programming practice, including gas optimization, code style, and etc.

We show the main concrete checkpoints in the following.

1.3.1 Security Issues

* Access control
* Permission management
* Whitelist and blacklist mechanisms
* Initialization consistency
* Improper use of the proxy system
* Reentrancy
* Denial of Service (DoS)
* Untrusted external call and control flow
* Exception handling
* Data handling and flow
* Events operation
* Error-prone randomness
* Oracle security
* Business logic correctness
* Semantic and functional consistency

* Emergency mechanism
* Economic and incentive impact

1.3.2 Additional Recommendation

* Gas optimization
* Code quality and style

Note *The previous checkpoints are the main ones. We may use more checkpoints during the auditing process according to the functionality of the project.*

1.4 Security Model

To evaluate the risk, we follow the standards or suggestions that are widely adopted by both industry and academy, including OWASP Risk Rating Methodology [2] and Common Weakness Enumeration [3]. The overall *severity* of the risk is determined by *likelihood* and *impact*. Specifically, likelihood is used to estimate how likely a particular vulnerability can be uncovered and exploited by an attacker, while impact is used to measure the consequences of a successful exploit.

In this report, both likelihood and impact are categorized into two ratings, i.e., *high* and *low* respectively, and their combinations are shown in Table 1.1.

Table 1.1: Vulnerability Severity Classification

Impact \ Likelihood	*High*	*Low*
High	High	Medium
Low	Medium	Low

Accordingly, the severity measured in this report are classified into three categories: **High**, **Medium**, **Low**. For the sake of completeness, **Undetermined** is also used to cover circumstances when the risk cannot be well determined.

Furthermore, the status of a discovered item will fall into one of the following five categories:

- **Undetermined** No response yet.
- **Acknowledged** The item has been received by the client, but not confirmed yet.
- **Confirmed** The item has been recognized by the client, but not fixed yet.
- **Partially Fixed** The item has been confirmed and partially fixed by the client.
- **Fixed** The item has been confirmed and fixed by the client.

[2] https://owasp.org/www-community/OWASP_Risk_Rating_Methodology

[3] https://cwe.mitre.org/

Chapter 2 Findings

In total, we found **three** potential security issues. Besides, we have **three** recommendations and **one** note.

- Low Risk: 3
- Recommendation: 3
- Note: 1

ID	Severity	Description	Category	Status
1	Low	Potential DoS due to a zero output amount in buy orders	Security Issue	Fixed
2	Low	Non-standard implementation of the assembly code in the `_delegateModule()` function	Security Issue	Fixed
3	Low	Inconsistent capacity check in the function `beforeSwap()`	Security Issue	Confirmed
4	-	Emit events in the function `setQuoter()`	Recommendation	Fixed
5	-	Remove redundant code	Recommendation	Fixed
6	-	Add validity checks for the input `quoter_` in the function `setQuoter()`	Recommendation	Fixed
7	-	Potential centralization risks	Note	-

The details are provided in the following sections.

2.1 Security Issue

2.1.1 Potential DoS due to a zero output amount in buy orders

Severity Low

Status Fixed in `Version 2`

Introduced by `Version 1`

Description In the contract `BigShortV2PolicyHook`, the function `beforeSwap()` caps an exact-input buy amount to the remaining launch capacity via the function `quoteNetCapacityUpTo()`. However, if the remaining capacity is too small, the capped input amount may result in an output of zero base tokens. Moreover, the transaction then reverts in the function `afterSwap()` due to the assertion `tokenDelta <= 0`. As a result, buy orders that potentially trigger the token graduation may fail due to a zero output amount.

```
293             uint256 net = (amount * poolBps) / BPS;
294             if (phase == LaunchPhase.Trading)
295                 net = lpVault.quoteNetCapacityUpTo(token, net);
296             if (net == 0) revert InvalidAmount();
297             s.poolAmount = net;
298             // Reserve tax AND unused terminal budget. Refund follows the Router executor/payer
                    .
```

Listing 2.1: src/v2/BigShortV2PolicyHook.sol

```
385          if (tokenDelta <= 0 || quoteDelta >= 0) revert InvalidAmount();
```

Listing 2.2: src/v2/BigShortV2PolicyHook.sol

```
151          (uint160 price,,,) = poolManager.getSlot0(lpVault.poolIdFor(p.token));
152          // A terminal launch buy is atomic with graduation. Keep the processing guard,
153          // pending context and incomplete receipt until all migration effects succeed.
154          // Errors (including exhausted gas) propagate and roll back this entire trade.
155          if (price == lpVault.terminalPriceFor(p.token)) {
156              lpVault.migrateInLock(p.token);
157              if (lpVault.phaseOf(p.token) != LaunchPhase.Graduated) revert MigrationIncomplete(p
                     .token);
158          }
159      }
160      if (!T.completeReceipt(p.nonce)) revert InvalidTradeContext();
```

Listing 2.3: src/v2/BigShortV2SettlementModule.sol

Impact Buy orders that potentially trigger the token graduation may fail due to a zero output amount.

Suggestion Revise the logic accordingly.

2.1.2 Non-standard implementation of the assembly code in the `_delegateModule()` function

Severity Low

Status Fixed in `Version 2`

Introduced by `Version 1`

Description In the contract `BigShortV2PolicyHook`, the function `_delegateModule()` copies the entire calldata and returndata into a memory region starting at offset 0 and marks this code block as `memory-safe`. According to the specification, assembly blocks marked as `memory-safe` may only access the defined spaces (e.g., memory regions at or beyond the free-memory pointer). However, the current implementation does not adhere to these established rules, breaking the definition of the `memory-safe` assembly blocks.

```
746   function _delegateModule(address implementation) private {
747       assembly ("memory-safe") {
748           calldatacopy(0, 0, calldatasize())
749           let ok := delegatecall(
750               gas(),
751               implementation,
752               0,
753               calldatasize(),
754               0,
755               0
756           )
757           returndatacopy(0, 0, returndatasize())
758           switch ok
759           case 0 {
```

```
760             revert(0, returndatasize())
761         }
762         default {
763             return(0, returndatasize())
764         }
765     }
766 }
```

Listing 2.4: src/v2/BigShortV2PolicyHook.sol

Impact The current implementation of the function `_delegateModule()` breaks the definition of the `memory-safe` assembly blocks.

Suggestion Use the free-memory pointer for the calldata copy.

2.1.3 Inconsistent capacity check in the function `beforeSwap()`

Severity Low

Status Confirmed

Introduced by `Version 1`

Description In the contract `BigShortV2PolicyHook`, the function `beforeSwap()` applies the capacity check on the exact-input buy path via the function `quoteNetCapacityUpTo()`. However, the exact-output buy path in the function `beforeSwap()` does not perform such checks. As a result, an exact-output buy order may revert before graduation because the pool has insufficient capacity.

```
292         if (s.exactInput) {
293             uint256 net = (amount * poolBps) / BPS;
294             if (phase == LaunchPhase.Trading)
295                 net = lpVault.quoteNetCapacityUpTo(token, net);
296             if (net == 0) revert InvalidAmount();
297             s.poolAmount = net;
298             // Reserve tax AND unused terminal budget. Refund follows the Router executor/payer
                    .
299             s.hookSpecified = amount - net;
300             _mintQuote(quote, s.hookSpecified - 1);
301         } else {
302             (s.poolAmount, s.burn, s.referral) = BigShortV2RouterMath
303                 .buyOutputPlan(amount, s.feeBps, ref != address(0));
304             s.hookSpecified = s.poolAmount - amount;
305             _mintBase(token, s.hookSpecified);
306         }
```

Listing 2.5: src/v2/BigShortV2PolicyHook.sol

Impact An exact-output buy beyond the remaining capacity reverts, so a valid trade that could trigger graduation fails.

Suggestion Revise the logic accordingly.

Feedback from the project The project stated that the revert is acceptable.

2.2 Recommendation

2.2.1 Emit events in the function `setQuoter()`

Status Fixed in `Version 2`

Introduced by `Version 1`

Description In the contract `BigShortV2PolicyHook`, the function `setQuoter()` does not emit the corresponding event compared to other setter functions. It is recommended to emit events in the function `setQuoter()`.

```
183   function setQuoter(address quoter_) external onlyOwner {
184       if (T.processing() || infinityVault.getLocker() != address(0))
185           revert TreasuryBusy();
186       if (
187           quoter_.code.length == 0 ||
188           IBigShortV2StandardQuoter(quoter_).vault() !=
189           address(infinityVault) ||
190           IBigShortV2StandardQuoter(quoter_).poolManager() !=
191           address(poolManager)
192       ) revert InvalidAddress();
193       quoter = quoter_;
194   }
```

Listing 2.6: src/v2/BigShortV2PolicyHook.sol

Suggestion Emit events in the function `setQuoter()`.

2.2.2 Remove redundant code

Status Fixed in `Version 2`

Introduced by `Version 1`

Description In the contract `BigShortV2PolicyHook`, there are several redundant errors. It is recommended to remove them for better code readability.

```
57   error UnauthorizedCallback();
58   error QuoteTransferMismatch();
59   error UnsupportedRoute();
60   error InvalidHookData();
61   error SellerAgeChanged();
62   error MigrationIncomplete(address token);
63   error TreasuryBusy();
64   error RewardNotificationFailed();
```

Listing 2.7: src/v2/BigShortV2PolicyHook.sol

Suggestion Remove the redundant code.

2.2.3 Add validity checks for the input `quoter_` in the function `setQuoter()`

Status Fixed in `Version 2`

Introduced by Version 1

Description In the contract BigShortV2PolicyHook, the function setQuoter() does not check whether the input quoter_ is not equal to the address router. It is recommended to add this check.

```
183   function setQuoter(address quoter_) external onlyOwner {
184       if (T.processing() || infinityVault.getLocker() != address(0))
185           revert TreasuryBusy();
186       if (
187           quoter_.code.length == 0 ||
188           IBigShortV2StandardQuoter(quoter_).vault() !=
189           address(infinityVault) ||
190           IBigShortV2StandardQuoter(quoter_).poolManager() !=
191           address(poolManager)
192       ) revert InvalidAddress();
193       quoter = quoter_;
194   }
```

Listing 2.8: src/v2/BigShortV2PolicyHook.sol

Suggestion Add validity checks for the input quoter_ in the function setQuoter().

2.3 Note

2.3.1 Potential centralization risks

Introduced by Version 1

Description In this project, several privileged roles (e.g., owner) can conduct sensitive operations, which introduces potential centralization risks. For example, the role owner can pause all trading across markets via the function setTradingPaused(). If the private keys of the privileged accounts are lost or maliciously exploited, it could pose a significant risk to the protocol.


BOOST WEB3 THROUGH NEXT-GENERATION SECURITY & USABILITY INNOVATIONS